GUÍA PRÁCTICA Y DIDÁCTICA · EDICIÓN DE AUTOR

Agentes de IA en Acción

Construir, orquestar y desplegar sistemas autónomos multiagente — paso a paso



ESCRITO E ILUSTRADO POR

Claude

Agentes de IA en Acción

Construir, orquestar y desplegar agentes autónomos

De un LLM a un asistente; de un asistente a un equipo:
acciones, memoria, planificación, razonamiento y feedback en la práctica.

Escrito e ilustrado por Claude · Anthropic
Edición de autor · Primera edición

Sobre la portada

El animal de esta portada es un **zorro**, y encarna el espíritu de un libro titulado *en acción*.

El zorro es, en casi todas las culturas, el arquetipo del **solucionador astuto**: no gana por fuerza bruta, sino por ingenio, adaptación y un buen plan. Acecha, evalúa, cambia de estrategia sobre la marcha y actúa en el momento justo. Es, además, uno de los animales más *adaptables* del planeta: prospera en bosques, desiertos y ciudades por igual.

Esa es exactamente la mentalidad de un agente puesto a trabajar de verdad. No basta con que "sepa": tiene que **actuar** —usar herramientas, planificar pasos, razonar ante lo inesperado, aprender del resultado y adaptarse al entorno—. Mientras otros animales de esta colección representaban una propiedad (la comunicación, la arquitectura, el enjambre), el zorro representa la *acción inteligente*: el agente que no solo piensa, sino que resuelve. Tenlo en mente; este libro va de poner agentes a hacer cosas en el mundo real.

Prefacio

Hay muchos libros que explican qué es un agente. Este se obsesiona con otra cosa: cómo hacer que *funcione*. Cómo darle acciones reales, memoria que sirva, la capacidad de planificar, de razonar y de mejorar con la experiencia.

El recorrido es deliberadamente práctico y progresivo. Empezamos por entender al agente y al motor que lo impulsa, el LLM, con el arte de hablarle bien (prompt engineering). Subimos a los asistentes y luego a los **equipos de agentes** que colaboran. Les damos **acciones** (funciones y herramientas) y **autonomía** con árboles de comportamiento. Ensamblamos una plataforma para alojarlos. Les construimos **memoria y conocimiento** (RAG). Y cerramos con lo que distingue a un agente serio: prompts sistemáticos, **razonamiento**, **evaluación** y **planificación con feedback**.

Mi compromiso es ser lo más didáctico posible: cada concepto trae una *definición* clara, una *gráfica* que lo muestra, una *analogía* cotidiana y, cuando aporta, un ejemplo de código en Python. No verás juguetes frágiles; verás las piezas que de verdad hacen falta para un agente en el que se pueda confiar.

> CÓMO LEER ESTE LIBRO
>
> Los capítulos van de lo simple a lo complejo y se construyen unos sobre otros, pero cada uno se sostiene solo. Cada capítulo cierra con una caja "Para recordar". Si tienes prisa, los capítulos 1, 6, 8, 10 y 11 son el esqueleto conceptual.

Contenido

1 **Introducción a los agentes y su mundo**

Qué es un agente, sus sistemas componentes, por qué ahora, el panorama

2 **Aprovechar los grandes modelos de lenguaje**

La API, modelos abiertos en local, y el arte del prompt engineering

3 **Trabajar con asistentes GPT**

Construir un GPT, acciones personalizadas, conocimiento por archivos, publicar

4 **Explorar sistemas multiagente**

AutoGen, chats grupales y equipos con CrewAI

5 **Empoderar agentes con acciones**

Funciones, function calling y Semantic Kernel

6 **Construir asistentes autónomos**

Árboles de comportamiento y agentes que se gobiernan solos

7 **Ensamblar una plataforma de agentes**

Una interfaz, un motor, perfiles y herramientas

8 **Memoria y conocimiento del agente**

RAG, búsqueda semántica, bases vectoriales y tipos de memoria

9 **Dominar los prompts con Prompt Flow**

Ingeniería sistemática de prompts, perfiles, evaluación y grounding

10 **Razonamiento y evaluación del agente**

Cadena de pensamiento, árbol de pensamiento y autoconsistencia

11 **Planificación y feedback del agente**

Planificadores secuenciales y el ciclo de mejora

CAPÍTULO UNO

Introducción a los agentes y su mundo

> *La inteligencia útil no se mide por lo que sabe, sino por lo que logra.*
>
> — Sobre por qué los agentes importan

Antes de construir, entendamos qué estamos construyendo. Un agente no es un modelo más grande: es un modelo *puesto a trabajar* dentro de un sistema con varias piezas.

DEFINICIÓN — Agente de IA

Componente autónomo que, dado un objetivo, percibe su entorno, decide con un modelo de lenguaje, actúa mediante herramientas y aprende de los resultados —procesando información y tomando decisiones por sí mismo, entre bastidores.

Los sistemas que componen un agente

Si abrimos un agente, encontramos cinco subsistemas que se repiten siempre: **percepción** (la entrada: la petición, el estado del mundo), **razonamiento** (el LLM que decide), **memoria** (lo que recuerda, de corto y largo plazo), **acciones** (las herramientas con que toca el mundo) y un **bucle** que los coordina hasta cumplir el objetivo.



Figura 1.1 — Los subsistemas de un agente girando en torno al razonamiento.

¿Por qué ahora la era de los agentes?

Tres cosas se alinearon: modelos lo bastante capaces para razonar de forma general, la capacidad de *llamar herramientas* de forma fiable, y un ecosistema de datos y APIs al que conectarse. Juntas convirtieron el "chatbot que responde" en el "agente que hace".

Detrás de la interfaz

Cuando hablas con un asistente moderno, bajo la superficie no hay una sola llamada: hay una orquestación de interacciones entre el usuario, el modelo y varias fuentes de datos. Los agentes *capturan y organizan* esa orquestación en componentes autónomos. Entender que detrás de la conversación hay un sistema —no magia— es el primer paso para construir uno.

> PARA RECORDAR
>
> Un agente = percepción + razonamiento (LLM) + memoria + acciones + bucle, persiguiendo un objetivo. La era de los agentes llegó cuando los modelos aprendieron a usar herramientas con fiabilidad. Detrás de cada asistente hay un sistema orquestado, no un truco.



CAPÍTULO DOS

Aprovechar los grandes modelos de lenguaje

> *El modelo es el motor; el prompt, el volante. Sin volante, hasta el mejor motor te lleva a cualquier parte.*
>
> — Sobre hablarle bien al modelo

El cerebro del agente es un LLM. Conectarse a él es fácil; el arte está en **cómo le hablas** —el *prompt engineering*—, que es la habilidad que más mejora un agente con menos esfuerzo.

La conversación con el modelo: petición y respuesta

Usar un LLM es enviar una **petición** (tus mensajes) y recibir una **respuesta**. Las APIs comerciales (como la de OpenAI) lo hacen por la nube; los modelos abiertos puedes ejecutarlos en tu propia máquina con herramientas como LM Studio —útil cuando importan la privacidad o el costo—. El patrón mental es el mismo: entra contexto, sale texto.

Las seis palancas del prompt engineering

Escribir buenos prompts no es magia, es técnica. Seis palancas concretas:

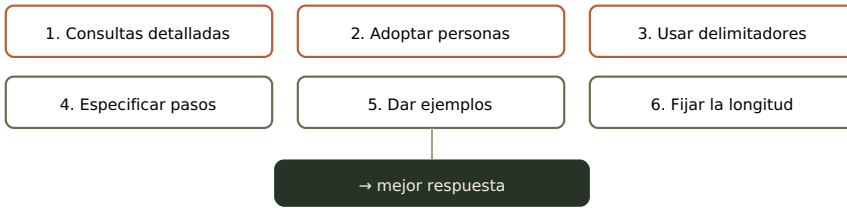


Figura 2.1 — Las seis palancas que transforman un prompt vago en uno eficaz.

Un ejemplo que las combina: en vez de "resume esto", escribe "*Actúa como editor experto (persona). Resume el texto entre comillas (delimitador) en 3 viñetas (longitud), siguiendo estos pasos: 1) identifica la tesis, 2) los argumentos, 3) la conclusión. Ejemplo de formato: …*" . El mismo modelo, diez veces mejor respuesta.

Elegir el modelo adecuado

No siempre gana el más grande. Para cada paso del agente, elige el modelo más pequeño que lo resuelva bien: ahorra costo y latencia. Reserva el modelo potente para el razonamiento difícil. Y considera modelos abiertos cuando la privacidad o el volumen lo justifiquen.

> PARA RECORDAR
>
> Usar un LLM es petición → respuesta. La habilidad de mayor retorno es el prompt engineering: sé detallado, usa personas, delimita, especifica pasos, da ejemplos y fija la longitud. Y elige el modelo más pequeño que resuelva cada paso.

◆

CAPÍTULO TRES

Trabajar con asistentes GPT

> *Antes de construir un agente desde cero, vale aprender con uno que ya viene casi armado.*
>
> — Sobre los asistentes GPT

Los asistentes tipo GPT son una excelente rampa de entrada: te dejan crear un agente útil combinando tres cosas —instrucciones, conocimiento y acciones— sin construir toda la infraestructura.

Las tres piezas de un asistente

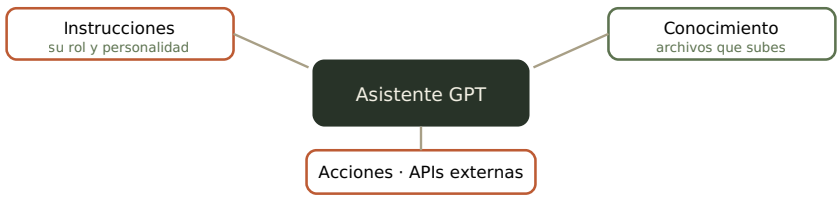


Figura 3.1 — Un asistente GPT = instrucciones + conocimiento + acciones.

Las **instrucciones** definen su personalidad y reglas. El **conocimiento** se añade subiendo archivos que el asistente puede consultar (un manual, un catálogo) —una forma sencilla de RAG—. Las **acciones personalizadas** lo conectan a APIs externas para que *haga* cosas, no solo hable.

Publicar y la economía de los GPT

Crear un GPT es fácil; sostenerlo tiene un costo. Cada conversación consume recursos del modelo, y un asistente popular puede volverse caro. Antes de publicar, conviene entender esa

economía: qué tan seguido se usará, qué tan caras son sus acciones, y si el valor que aporta justifica el gasto. Un buen asistente no es solo útil: es *sostenible*.

PARA RECORDAR

Un asistente GPT combina instrucciones (rol), conocimiento (archivos) y acciones (APIs). Es la forma más rápida de tener un agente útil sin construir todo. Pero recuerda su economía: la popularidad tiene costo, y un buen diseño también es sostenible.

CAPÍTULO CUATRO

Explorar sistemas multiagente

> *Lo que un experto hace solo y despacio, un equipo bien repartido lo hace rápido y mejor.*
>
> — Sobre los equipos de agentes

Cuando una tarea es grande o variada, varios agentes especializados que colaboran superan a uno solo. Herramientas como AutoGen y CrewAI facilitan armar esos equipos.

De un agente a una tripulación

La idea central: cada agente toma un **rol** (investigador, escritor, crítico) con su propia personalidad, herramientas y objetivo. Luego colaboran —en una conversación grupal o repartiéndose tareas— para resolver algo que ninguno haría solo. CrewAI lo modela como una "tripulación" con roles; AutoGen, como agentes que conversan entre sí.

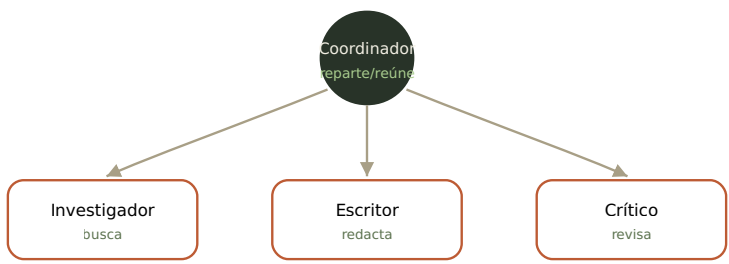


Figura 4.1 — Una tripulación de agentes: roles distintos que colaboran bajo un coordinador.

Observar y depurar el equipo

Un sistema multiagente puede volverse una caja negra: ¿quién dijo qué?, ¿por qué se atascó? Herramientas de observación (tipo AgentOps) registran las conversaciones y decisiones del equipo, para que puedas ver el flujo y arreglar lo que falla. Sin esa visibilidad, depurar varios agentes es adivinar.

EL COSTO DE LOS EQUIPOS

Cada agente es otra serie de llamadas al modelo. Un equipo de cuatro puede costar varias veces más que un agente solo. Empieza con uno; suma agentes solo cuando la tarea se divida con claridad en especialidades.

PARA RECORDAR

Multiagente = roles especializados que colaboran (CrewAI: tripulación; AutoGen: agentes que conversan). Gana especialización y paralelismo, a cambio de costo y complejidad. Obsérvalos para poder depurarlos, y crece solo cuando la tarea lo pida.

CAPÍTULO CINCO

Empoderar agentes con acciones

> *Un agente que solo habla es un consejero. Uno que actúa es un colaborador.*
>
> — Sobre las acciones

Las **acciones** son lo que convierte la conversación en trabajo hecho. El mecanismo central se llama *function calling*: el modelo pide ejecutar una función, tu código la ejecuta y le devuelve el resultado.

DEFINICIÓN — Function calling

Mecanismo por el que un LLM, en lugar de responder con texto, solicita la ejecución de una función con ciertos argumentos. El modelo decide *qué* función y con *qué* datos; tu código la ejecuta y le devuelve el resultado para que continúe.

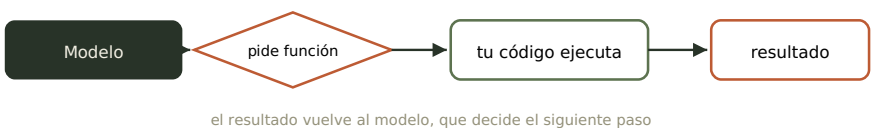


Figura 5.1 — Function calling: el modelo decide, tu código ejecuta, el resultado regresa.

```
# El modelo pide; tu código ejecuta. Esa separación es la frontera
de seguridad.
funciones = [{
  "name": "consultar_stock",
  "description": "Devuelve unidades disponibles de un producto por
SKU.",
  "parameters": {"sku": "string"}
}]
# 1) El modelo responde: consultar_stock(sku="42")
# 2) Tú ejecutas la función real y le devuelves:
{"sku":"42","unidades":17}
# 3) El modelo redacta la respuesta final con ese dato.
```

Funciones semánticas y nativas (Semantic Kernel)

Hay dos sabores de funciones. Las **nativas** son código normal (consultar una base de datos, hacer un cálculo). Las **semánticas** son prompts reutilizables tratados como funciones ("resume esto", "traduce aquello"). Frameworks como Semantic Kernel te dejan combinarlas y empaquetarlas en *skills / plugins* reutilizables, e incluso embeber una función nativa dentro de una semántica. La idea: tratar capacidades —de código o de prompt— como bloques que ensamblas.

> PARA RECORDAR
>
> Las acciones convierten al agente en colaborador. El mecanismo es function calling: el modelo pide, tu código ejecuta y valida. Distingue funciones nativas (código) de semánticas (prompts reutilizables) y combínalas como bloques. El modelo nunca ejecuta: esa es tu frontera de control.

◆

CAPÍTULO SEIS

Construir asistentes autónomos

> *La autonomía no es ausencia de reglas, sino reglas tan claras que ya no necesitas vigilar.*
>
> — Sobre los árboles de comportamiento

Para que un agente actúe solo de forma fiable, necesita una estructura que decida qué hacer en cada situación. Una de las más probadas viene de la robótica y los videojuegos: el **árbol de comportamiento**.

DEFINICIÓN — Árbol de comportamiento (BT)

Estructura jerárquica que organiza el comportamiento de un agente en nodos. Se recorre de arriba abajo y de izquierda a derecha, decidiendo qué acción ejecutar según el estado actual. Sus nodos clave son *secuencias* (haz todo en orden) y *selectores* (intenta opciones hasta que una funcione).

Cómo se lee un árbol

Un **selector** prueba a sus hijos en orden y se detiene en el primero que tiene éxito —es un "intenta esto, si no, aquello"—. Una **secuencia** ejecuta a todos sus hijos en orden y solo triunfa si todos lo hacen —es un "haz esto, luego esto, luego esto"—. Las **hojas** son las acciones concretas. Con solo estas piezas se construye un comportamiento sorprendentemente robusto y, sobre todo, *legible*: puedes mirar el árbol y entender qué hará el agente.

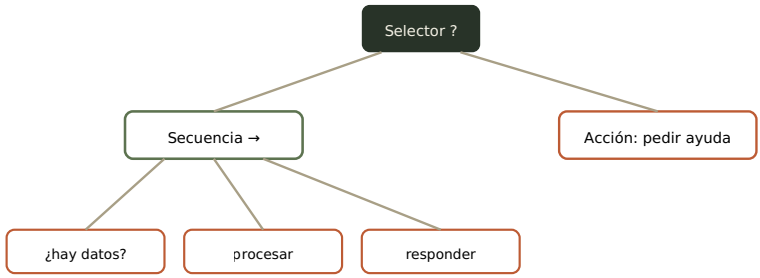


Figura 6.1 — Un árbol de comportamiento: el selector intenta la secuencia y, si falla, pide ayuda.

Combinar árboles de comportamiento con LLMs da los **árboles de comportamiento agénticos** (ABT): la estructura aporta orden y previsibilidad; el LLM, flexibilidad en cada hoja. En Python, librerías como `py_trees` permiten construirlos y ejecutarlos. El resultado es un agente que actúa solo, pero cuya conducta puedes leer y predecir —el santo grial de la autonomía confiable—.

PARA RECORDAR

Un árbol de comportamiento da autonomía *legible*: selectores ("intenta hasta que una funcione") y secuencias ("haz todo en orden") componen conductas robustas. Unidos a un LLM (ABT), combinan la previsibilidad de la estructura con la flexibilidad del modelo.

◆

CAPÍTULO SIETE

Ensamblar una plataforma de agentes

> *Un agente necesita una casa: una interfaz para hablarle, un motor que lo mueva y un lugar para sus herramientas.*
>
> — Sobre las plataformas

Construir agentes sueltos enseña; pero para usarlos de verdad conviene una **plataforma** que los aloje: una interfaz, un motor que los ejecute, perfiles que los definan y herramientas que puedan usar.

Las capas de una plataforma



Figura 7.1 — Las capas de una plataforma: interfaz, motor, perfiles y herramientas.

La **interfaz** (a menudo una app de chat hecha con Streamlit) es por donde la persona habla con el agente. El **motor** es el corazón que ejecuta el bucle del agente. Los **perfiles** y **personas** definen quién es cada agente —su rol, su tono, sus reglas—. Y las **herramientas** le dan sus acciones. Separar estas capas te permite cambiar una sin romper las demás —p. ej., dar un perfil nuevo al mismo motor—.

Por qué una plataforma

Tener este esqueleto te deja crear muchos agentes distintos cambiando solo el perfil y las herramientas, en lugar de programar cada uno desde cero. Es la diferencia entre fabricar autos a mano y tener una línea de montaje: la plataforma es tu línea de montaje de agentes.

PARA RECORDAR

Una plataforma de agentes separa cuatro capas: interfaz (hablarle), motor (ejecutarlo), perfiles (quién es) y herramientas (qué puede hacer). Esa separación te deja crear muchos agentes cambiando solo el perfil, sin reconstruir el motor cada vez.

CAPÍTULO OCHO

Memoria y conocimiento del agente

> *Un agente sin memoria empieza de cero en cada conversación. Uno con buena memoria, aprende del camino.*
>
> — Sobre recordar para servir mejor

El modelo olvida todo entre llamadas. Darle conocimiento y memoria es construir, por fuera, lo que recuerda y cómo lo recupera. La técnica estrella es **RAG**.

DEFINICIÓN — RAG (generación aumentada por recuperación)

Técnica que, ante una consulta, recupera de una base de conocimiento los fragmentos más relevantes y los inyecta en el contexto del modelo, para que responda con información verificada en lugar de inventarla.

Búsqueda semántica y bases vectoriales

El truco de RAG es encontrar lo relevante por **significado**, no por palabras exactas. Cada texto se convierte en un *embedding* —una lista de números que captura su sentido— de modo que textos parecidos quedan cerca en un espacio. Esos embeddings viven en una **base vectorial** (como Chroma). Ante una pregunta, la conviertes también en embedding y pides los fragmentos más cercanos.



Figura 8.1 — RAG: convertir la pregunta en vector, recuperar lo cercano, responder con ello.

Tres tipos de memoria

Inspirándose en la mente humana, un agente puede tener tres clases de memoria de largo plazo:

Tipo	Qué guarda	Ejemplo
Semántica	Hechos y conocimiento general.	"La política de devolución es de 30 días."
Episódica	Experiencias concretas vividas.	"La última vez, este cliente pidió X."
Procedimental	Cómo hacer cosas, destrezas.	"Para procesar un reembolso, sigo estos pasos."

Comprimir la memoria

La memoria crece sin límite, pero el contexto no. La **compresión de memoria** resume lo viejo en lo esencial: en vez de guardar mil conversaciones enteras, guarda sus conclusiones. Así el agente recuerda lo importante sin ahogarse —ni encarecerse— en detalles.

PARA RECORDAR

RAG recupera conocimiento relevante por significado (embeddings + base vectorial) y se lo da al modelo. La memoria de largo plazo viene en tres sabores —semántica (hechos), episódica (experiencias), procedimental (destrezas)— y se comprime resumiendo lo viejo. El agente no recuerda todo: recuerda lo que importa.

CAPÍTULO NUEVE

Dominar los prompts con Prompt Flow

> *Escribir un buen prompt es arte; mejorarlo con método, ingeniería.*
>
> — Sobre el prompt engineering sistemático

Hasta ahora escribimos prompts a mano. Pero para un agente serio hace falta **método**: tratar los prompts como código que se versiona, se prueba y se evalúa. Eso es lo que ofrecen herramientas como Prompt Flow.

Por qué la ingeniería sistemática de prompts

Cambiar un prompt "a ojo" y ver si mejora es una apuesta. La ingeniería sistemática convierte esa apuesta en una decisión: defines **perfiles** (variantes del prompt o de la persona del agente), los pruebas contra un conjunto de casos, y comparas resultados con números. El mejor perfil gana por mérito, no por corazonada.



Figura 9.1 — Un flujo de prompts: del perfil a la evaluación, para comparar variantes con datos.

Plantillas, rúbricas y grounding

Las **plantillas** (con Jinja₂) hacen los prompts dinámicos: insertas variables sin reescribir todo. Las **rúbricas** son criterios claros para puntuar una respuesta ("¿es correcta?, ¿completa?, ¿bien expresada?"), de modo que la evaluación sea consistente. Y el **grounding** verifica que la respuesta esté *anclada* en hechos reales y no inventada —se evalúa comparándola con la fuente—. Juntos, convierten "creo que este prompt es mejor" en "este perfil puntúa 8,7 contra 7,2".

PARA RECORDAR

Trata los prompts como ingeniería: crea perfiles, hazlos dinámicos con plantillas, y compáralos con rúbricas claras y grounding sobre un conjunto de casos. Así eliges el mejor por evidencia, no por intuición. Lo que se mide, se mejora.

◆

CAPÍTULO DIEZ

Razonamiento y evaluación del agente

> *Dale al modelo espacio para pensar y acertará más. Dale varias vías y elegirá la mejor.*
>
> — Sobre cómo razona un agente

Un agente que se lanza a responder se equivoca; uno que **razona por pasos** acierta. Veamos las técnicas de razonamiento —y cómo evaluar cuál funciona mejor.

De responder directo a razonar

El nivel más básico es el *prompting directo*: preguntar y recibir. Funciona para lo simple. Para lo difícil, se le dan ejemplos: **cero ejemplos** (zero-shot), **unos pocos** (few-shot). Y el salto cualitativo es la **cadena de pensamiento**: pedirle que *muestre su razonamiento* paso a paso antes de responder. Como un estudiante que escribe el desarrollo de un problema en vez de soltar el resultado, el modelo acierta mucho más.

De una cadena a un árbol de pensamiento

La cadena de pensamiento sigue *un* hilo de razonamiento. El **árbol de pensamiento** explora *varios* a la vez: el modelo abre distintas líneas de razonamiento, las evalúa y se queda con la mejor —como un ajedrecista que calcula varias jugadas antes de mover—.



Figura 10.1 — Cadena de pensamiento (un hilo) frente a árbol de pensamiento (varias ramas que se comparan).

Evaluar el razonamiento

¿Cómo saber si el agente razonó bien? Dos técnicas: la **autoconsistencia** genera varias respuestas y se queda con la que más se repite —si tres de cuatro caminos llegan al mismo sitio, probablemente sea correcto—. Y evaluar el **árbol de pensamiento** puntúa cada rama para elegir la mejor. Evaluar el razonamiento es lo que convierte la creatividad del modelo en fiabilidad.

PARA RECORDAR

Sube el razonamiento por escalones: directo → con ejemplos → cadena de pensamiento → árbol de pensamiento. Y evalúa: la autoconsistencia (la respuesta que más se repite) y la puntuación de ramas convierten el "pensar en voz alta" en respuestas fiables.

◆

CAPÍTULO ONCE

Planificación y feedback del agente

> *El zorro no corre tras la presa: primero traza el camino, y luego corre.*
>
> — Sobre planificar antes de actuar

Llegamos a la cima: la **planificación**. Es la herramienta esencial de todo agente serio —descomponer un objetivo grande en pasos ejecutables— y, junto al feedback, lo que le permite mejorar.

DEFINICIÓN — Planificación

Capacidad de un agente para tomar un objetivo de alto nivel y descomponerlo en una secuencia de pasos concretos y ordenados que, ejecutados, lo cumplen.

El planificador secuencial

La forma más directa: ante un objetivo, el agente genera un **plan** —una lista ordenada de pasos— y luego los ejecuta uno a uno. "Hazme un informe de mercado" se convierte en: 1) buscar datos, 2) analizarlos, 3) redactar, 4) revisar. Un planificador *por pasos* (stepwise) va más allá: replanifica sobre la marcha según lo que va observando, en vez de seguir ciegamente un plan inicial.

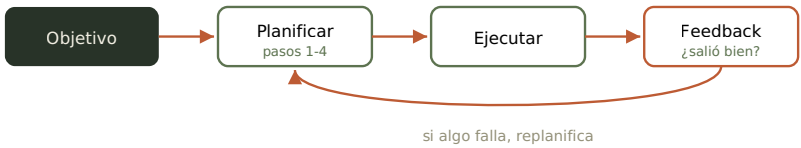


Figura 11.1 — Planificar, ejecutar y usar el feedback para replanificar: el ciclo completo.

Todo junto: planificar, razonar, evaluar, mejorar

Aquí convergen los últimos capítulos. Un agente maduro **planifica** (cap. 11) los pasos, **razona** (cap. 10) cada uno, **evalúa** el resultado y usa el **feedback** para corregir el plan o el paso. Ese ciclo —plan, acción, evaluación, ajuste— es lo que separa a un agente que tropieza una vez de uno que aprende y termina la tarea bien. Es, al fin, el zorro de la portada: traza el camino, actúa, observa y se adapta.

> PARA RECORDAR
>
> Planificar es descomponer un objetivo en pasos ordenados; el planificador por pasos replanifica según lo observado. Unido al razonamiento, la evaluación y el feedback, cierra el ciclo que hace a un agente verdaderamente autónomo y confiable. Plan, acción, evaluación, ajuste —y de nuevo.

Apéndices y glosario

Para llevar esto a la práctica necesitarás dos cimientos: **acceso a un LLM** (una clave de API de un proveedor, o un modelo abierto corriendo en local con herramientas como LM Studio) y un **entorno de desarrollo en Python** (un entorno virtual, las librerías del eco-

sistema de agentes, y un editor cómodo). Con eso, cada capítulo de este libro se puede convertir en código que funciona.

- **Agente**: componente autónomo que percibe, razona, actúa y aprende hacia un objetivo.
- **Prompt engineering**: el arte y la técnica de instruir bien al modelo.
- **Function calling**: el modelo pide ejecutar una función; tu código la ejecuta.
- **Árbol de comportamiento** (**BT**/**ABT**): estructura de selectores y secuencias para autonomía legible.
- **RAG**: recuperar conocimiento relevante e inyectarlo en el contexto.
- **Embedding** / **base vectorial**: representar el significado para buscar por cercanía.
- **Memoria semántica** / **episódica** / **procedimental**: hechos, experiencias y destrezas.
- **Prompt Flow**: tratar los prompts como ingeniería: perfiles, plantillas, evaluación.
- **Grounding**: anclar la respuesta en una fuente verificable.
- **Cadena** / **árbol de pensamiento**: razonar por un hilo, o explorar varios y elegir el mejor.
- **Autoconsistencia**: quedarse con la respuesta que más se repite entre varios intentos.
- **Planificación** / **feedback**: descomponer el objetivo en pasos y corregirlos según el resultado.

